

Redes Neuronales y Aprendizaje Profundo

Q3 2026

Ing. Iván de Jesús Deras Tábor

Laboratorio: De Redes Densas a CNNs

MNIST, Fashion-MNIST, y el Caso a Favor de las Convoluciones

Objetivos de Aprendizaje

Al finalizar este laboratorio, deberías ser capaz de:

- Construir y entrenar una red neuronal densa (totalmente conectada) simple para clasificación de imágenes.
- Demostrar empíricamente que la misma arquitectura se desempeña de forma muy distinta según qué tan difícil sea la tarea subyacente.
- Explicar por qué aplanar imágenes para una red densa descarta información que las capas convolucionales sí pueden aprovechar.
- Construir y entrenar una Red Neuronal Convolucional (CNN) y mostrar cómo recupera el desempeño que la red densa no pudo lograr.
- Usar dropout como técnica de regularización y evaluar si mejora la generalización.
- Sacar conclusiones basadas en evidencia a partir de curvas de precisión y pérdida, en lugar de un único valor de precisión.

Hoja de Ruta del Laboratorio

Este laboratorio está estructurado como un experimento de tres pasos, no simplemente como tres ejercicios separados. Ten en mente la pregunta general mientras avanzas: ¿una brecha de desempeño es causada por la arquitectura, o por la dificultad de la tarea en sí misma? Solo puedes responder eso manteniendo fijo uno de los dos factores mientras cambias el otro — que es exactamente lo que hacen las Partes 1 y 2.

Paso	Qué harás
Parte 1	Red densa en MNIST (dígitos) — establece un modelo de referencia.
Parte 2	La misma arquitectura densa exacta, reentrenada desde cero en Fashion-MNIST (ropa) — mismo modelo, tarea más difícil.
Parte 3	Una CNN, entrenada en Fashion-MNIST — misma tarea difícil, arquitectura mejor adaptada.

¿Por qué reentrenar desde cero en la Parte 2? No estamos reutilizando los pesos aprendidos en MNIST. El objetivo es ver qué tan bien esta arquitectura, partiendo desde cero, puede aprender Fashion-MNIST por sus propios méritos — no probar transfer learning. Reutilizar los pesos de MNIST respondería una pregunta distinta ("¿las características de los dígitos se transfieren a la ropa?") de la que este laboratorio plantea ("¿es la arquitectura, o la tarea, el factor limitante?").

Parte 1 — Modelo de Referencia: Red Densa en MNIST

MNIST es un dataset de 70,000 imágenes en escala de grises de dígitos escritos a mano (0-9), cada una de 28x28 píxeles.

1.1 Cargando los Datos

```
from dotenv import load_dotenv
import os

load_dotenv('.env')
mnist_train_fp = os.getenv('MNIST_TRAIN_FILEPATH')
mnist_test_fp = os.getenv('MNIST_TEST_FILEPATH')

mnist_train = np.load(mnist_train_fp)
mnist_test = np.load(mnist_test_fp)

print(f"Train dataset file path: {mnist_train_fp}")
print(f"Test dataset file path: {mnist_test_fp}")
```

Nota: `mnist_train` y `mnist_test` son básicamente un diccionarios con 2 llaves **images** y **labels**. El valor asociado a la llave **images** son las imagenes con dimensión 28x28 y el valor asociado a la llave **labels** son las etiquetas correspondientes a cada imagen.

Adicionalmente para que este código funcione deberá crearse un archivo llamado `.env` en el mismo folder del notebook o archivo de python. El archivo deberá tener dos variables de entorno:

```
MNIST_TRAIN_FILEPATH=<ruta del archivo mnist de entrenamiento>
MNIST_TEST_FILEPATH=<ruta del archivo mnist para pruebas>
```

TAREA — 1.2 — Divide y Escala

- Usa `train_test_split` con `stratify=y` y un `random_state` fijo (usa el mismo valor durante todo este laboratorio, por ejemplo 42, para que cada comparación que hagas sea equivalente).

Nota: La función `train_test_split` está disponible en la módulo `sklearn.model_selection`

- Escala los valores de los píxeles al rango `[0, 1]` dividiendo entre 255.0 — aquí no se necesita `StandardScaler`.
- Establece también `torch.manual_seed(...)` una sola vez, con el mismo valor, antes de construir o entrenar cualquier modelo en este laboratorio.

1.3 Especificación de la Arquitectura

Capa	Especificación
Entrada	784 características (imagen de 28x28 aplanada)
Capa 1 (oculta)	Linear(784 -> 128), seguida de ReLU
Capa 2 (salida)	Linear(128 -> 10), logits crudos (sin activación)

¿Por qué 10 neuronas de salida, y sin sigmoid en la salida?

MNIST tiene 10 clases posibles, así que la red necesita una neurona de salida por clase, cada una produciendo un puntaje crudo (logit). Para problemas multiclase no agregamos Softmax dentro del modelo — `nn.CrossEntropyLoss` espera logits crudos directamente y aplica internamente el equivalente de softmax, en una operación combinada numéricamente más estable.

TAREA — 1.4 — Construye, Entrena y Evalúa

```
class DenseNet(nn.Module):
    def __init__(self):
        super(DenseNet, self).__init__()
        self.layer1 = nn.Linear(____, ____ )
        self.layer2 = nn.Linear(____, ____ )

    def forward(self, x):
        output = ____ (self.layer1(x))    # aplica la activación correcta
        output = self.layer2(output)     # sin activación aquí – ¿por qué?
        return output
```

- Pérdida: usa la función de pérdida adecuada para logits crudos + etiquetas de clase enteras (ver la pista más abajo si tienes dudas).
- Tu tensor de etiquetas (y) debe ser un LongTensor de índices de clase (0-9) — sin `unsqueeze`, sin codificación one-hot.
- Para obtener la clase predicha a partir de la salida del modelo, usa `torch.argmax(preds, dim=1)`.
- Registra la pérdida de entrenamiento Y de validación por época, y grafica ambas curvas — reutiliza el patrón de tu notebook de cáncer de mama.

Pista: Quieres una función de pérdida que reciba logits crudos (no probabilidades) y etiquetas de clase enteras (no vectores one-hot) directamente. Revisa la documentación de PyTorch para funciones de pérdida diseñadas para clasificación multiclase.

Registra este número: Anota tu precisión de prueba en MNIST. La compararás directamente con la Parte 2 — misma arquitectura, dataset distinto.

Parte 2 — La Misma Red, una Tarea Más Difícil: Fashion-MNIST

Fashion-MNIST es un reemplazo directo de MNIST con el mismo tamaño de imagen (28x28 en escala de grises) y el mismo número de clases (10) — pero en lugar de dígitos, cada imagen es una prenda de ropa (camiseta, pantalón, suéter, vestido, abrigo, sandalia, camisa, tenis, bolso, botín).

Por qué esta es una comparación justa: Misma forma de entrada, mismo número de clases, misma arquitectura, mismos hiperparámetros, misma semilla. Lo único que cambia es qué representan

realmente las imágenes. Cualquier diferencia de precisión que observes es entonces atribuible a la dificultad de la tarea, no a algún cambio que hayas hecho en el modelo.

TAREA — 2.1 — Carga Fashion-MNIST

```
from dotenv import load_dotenv
import os

load_dotenv('.env')
fmnist_train_fp = os.getenv('FMNIST_TRAIN_FILEPATH')
fmnist_test_fp = os.getenv('FMNIST_TEST_FILEPATH')

fmnist_train = np.load(mnist_train_fp)
fmnist_test = np.load(mnist_test_fp)

print(f"Fashion-Mnist train dataset file path: {fmnist_train_fp}")
print(f"Fashion-Mnist test dataset file path: {fmnist_test_fp}")
```

- Aplica exactamente el mismo procedimiento aplicado en la Parte 1. Ten en cuenta que las variables de entorno tienen nombre diferente al de la Parte 1. FMNIST_TRAIN_FILEPATH y FMNIST_TEST_FILEPATH. Puedes utilizar el mismo archivo .env de la Parte 1.

TAREA — 2.2 — Reentrena la Misma Arquitectura desde Cero

Crea una nueva instancia de la misma clase DenseNet de la Parte 1 (mismos tamaños de capa, misma activación) y entrénala en Fashion-MNIST usando la misma función de pérdida, optimizador, tasa de aprendizaje, tamaño de batch, y número de épocas que en la Parte 1.

- No reutilices los pesos de tu modelo de MNIST — este debe ser un modelo recién inicializado, entrenado únicamente con datos de Fashion-MNIST.
- Registra la pérdida de entrenamiento y validación por época, exactamente como antes, y grafica ambas curvas.
- Calcula la precisión de prueba de la misma forma que en la Parte 1 (torch.argmax, y luego compara con las etiquetas verdaderas).

TAREA — 2.3 — Compara la Parte 1 vs. la Parte 2

Completa una tabla como esta en tu reporte:

Dataset	Precisión de Prueba (misma arquitectura)
MNIST (Parte 1)	
Fashion-MNIST (Parte 2)	

Pregunta de discusión: Si la precisión en Fashion-MNIST es notablemente menor que en MNIST, ¿qué te dice eso — y qué descarta? Dado que la arquitectura, los hiperparámetros, y la semilla son idénticos, ¿puede explicarse la caída porque el modelo sea "peor"? ¿Qué debe ser diferente entonces?

Pregunta de discusión: Observa tus dos curvas de pérdida una junto a la otra. ¿El modelo de Fashion-MNIST alcanza una pérdida de entrenamiento tan baja como la del modelo de MNIST? ¿Qué sugiere eso sobre qué tan bien una red densa aplanada, basada solo en píxeles, puede representar formas de ropa en comparación con formas de dígitos?

Parte 3 — Mejorando la Tarea Más Difícil con una CNN

Ahora pondrás a prueba si una arquitectura mejor adaptada — no más entrenamiento, no más datos — puede cerrar la brecha que acabas de medir en Fashion-MNIST.

3.1 Por Qué las Redes Densas No Son Adecuadas para Imágenes

El primer paso de tu red densa fue aplanar la imagen de 28x28 en un vector de longitud 784. Esto descarta algo importante: la estructura espacial. El píxel 15 y el píxel 43 podrían estar justo al lado uno del otro en la imagen original, o podrían estar muy alejados — una vez aplanados en un vector 1D, la red no tiene forma de saber qué píxeles eran vecinos.

Una capa densa trata cada píxel como una característica independiente y sin orden. Pero las imágenes no funcionan así — los píxeles cercanos están altamente correlacionados (forman bordes, curvas y formas juntos), y el mismo patrón (digamos, el cuello de una camisa, o la suela de un zapato) puede aparecer en cualquier parte de la imagen, en posiciones y escalas ligeramente distintas entre fotos. Una red densa tiene que aprender ese patrón por separado para cada posición posible, ya que no tiene una noción incorporada de "esta misma forma, solo desplazada". Las prendas en Fashion-MNIST varían más en su disposición exacta de píxeles que los dígitos centrados y normalizados — lo cual es una razón plausible de que la red densa haya tenido más dificultades aquí que en MNIST.

3.2 Qué Hacen Diferente las Capas Convolucionales

Una capa convolucional (`nn.Conv2d`) desliza un pequeño filtro aprendible (también llamado kernel — típicamente de 3x3 o 5x5 píxeles) a través de la imagen, calculando una suma ponderada en cada posición. Esto tiene dos propiedades cruciales que las capas densas no tienen:

Conectividad local

Cada valor de salida depende solo de un pequeño vecindario de píxeles de entrada (del tamaño del kernel), no de toda la imagen a la vez. Esto coincide con la intuición de que si un píxel forma parte de un borde o una curva depende principalmente de sus vecinos inmediatos, no de un píxel en la esquina opuesta de la imagen.

Compartición de parámetros (invariancia a la traslación)

El mismo filtro (el mismo conjunto de pesos) se aplica en cada posición de la imagen. Si la red aprende un filtro que detecta un borde diagonal, ese filtro puede detectar un borde diagonal en cualquier parte de la imagen — arriba a la izquierda, en el centro, abajo a la derecha — sin necesitar pesos separados para cada ubicación. Por esto las CNNs generalizan mucho mejor en imágenes, con muchos menos parámetros de los que necesitaría una red densa equivalente, y por qué tienden a ser más robustas ante una forma que aparece ligeramente desplazada o a otra escala — justo el tipo de variación que Fashion-MNIST tiene en mayor medida que MNIST.

Mapas de características y múltiples filtros

Una sola capa `Conv2d` típicamente aprende muchos filtros a la vez (por ejemplo, 16 o 32), cada uno especializándose en detectar un patrón simple distinto (un borde en cierto ángulo, una esquina, una mancha de cierta forma). La salida es una pila de "mapas de características" — uno por filtro — cada uno resaltando dónde se detectó ese patrón particular en la imagen.

Capas de pooling

Una capa de pooling (comúnmente `nn.MaxPool2d`) reduce el muestreo de los mapas de características tomando, por ejemplo, el valor máximo en cada pequeña región (por ejemplo, 2×2). Esto reduce el tamaño espacial de los datos a medida que avanzan hacia capas más profundas, reduce el cómputo, y agrega una pequeña cantidad de invariancia a la traslación (una característica ligeramente desplazada produce una salida agrupada similar).

La idea general: Las primeras capas convolucionales tienden a aprender características simples y generales (bordes, esquinas, texturas). Las capas más profundas combinan estas en patrones cada vez más complejos y específicos de la tarea (cuellos, tirantes, suelas, y eventualmente siluetas completas de prendas). Las capas densas al final de la red luego combinan estas características de alto nivel en una decisión final de clasificación.

3.3 Arquitectura CNN para Este Laboratorio

Capa	Especificación
Entrada	1 x 28 x 28 (1 canal, escala de grises)
Capa Convolucional 1	Conv2d(1 -> 16 canales, kernel_size=3, padding=1), luego ReLU
Pooling	MaxPool2d(kernel_size=2) — reduce el tamaño espacial a la mitad, a 14 x 14
Capa Convolucional 2	Conv2d(16 -> 32 canales, kernel_size=3, padding=1), luego ReLU
Pooling	MaxPool2d(kernel_size=2) — reduce el tamaño espacial a la mitad, a 7 x 7
Aplanado (Flatten)	Reorganiza los mapas de características de 32 x 7 x 7 en un solo vector (32*7*7 = 1568)
Capa Densa (salida)	Linear(1568 -> 10), logits crudos

TAREA — 3.4 — Construye la CNN

Completa la clase CNN a continuación. La entrada a este modelo NO debe estar aplanada de antemano — Conv2d espera un tensor 4D de forma (batch_size, canales, altura, anchura).

```
class ConvNet(nn.Module):
    def __init__(self):
        super(ConvNet, self).__init__()
        self.conv1 = nn.Conv2d(in_channels=__, out_channels=__, kernel_size=3,
padding=1)
        self.conv2 = nn.Conv2d(in_channels=__, out_channels=__, kernel_size=3,
padding=1)
        self.pool = nn.MaxPool2d(kernel_size=2)
        self.fc_out = nn.Linear(__, 10)

    def forward(self, x):
        # forma de x: (batch_size, 1, 28, 28)
        output = self.pool(F.relu(self.conv1(x)))
        output = __ (F.relu(self.conv2(output))) # aplica pooling nuevamente
        output = output.view(output.size(0), -1) # aplanar para la capa densa
        output = self.fc_out(output)
        return output
```

Recordatorio sobre el reordenamiento: Tus tensores de Fashion-MNIST de la Parte 2 tienen forma (n_samples, 784). Antes de pasarlos a ConvNet, reorganízalos de vuelta a forma de imagen: `x.view(-1, 1, 28, 28)` — dimensión de batch inferida, 1 canal, tamaño espacial 28x28.

TAREA — 3.5 — Entrena, Evalúa y Compara los Tres Resultados

Entrena la CNN en Fashion-MNIST usando la misma función de pérdida, configuración de optimizador, semilla, y patrón de ciclo de entrenamiento que en las Partes 1 y 2. Luego completa esta comparación de tres vías:

Modelo / Dataset	Precisión de Prueba	Cantidad de Parámetros
Densa en MNIST (Parte 1)		
Densa en Fashion-MNIST (Parte 2)		
CNN en Fashion-MNIST (Parte 3)		

- Cantidad de parámetros: usa `sum(p.numel() for p in model.parameters())` en el DenseNet y el ConvNet — ¿cuál tiene más parámetros?

Pregunta de discusión: ¿Cuánto de la brecha de precisión entre la Parte 1 y la Parte 2 logró recuperar la CNN en la Parte 3? ¿La cerró por completo, parcialmente, o no la cerró en absoluto?

Pregunta de discusión: Si la CNN tiene menos parámetros que la red densa pero logra mayor precisión en Fashion-MNIST, ¿qué te dice eso sobre el valor de incorporar los supuestos estructurales correctos (conectividad local, compartición de parámetros) en una arquitectura, en lugar de simplemente agregar más capacidad bruta?

3.6 Más Allá de la Precisión (Accuracy): Precision, Recall y F1-Score

Un único valor de accuracy trata cada clase por igual y oculta cómo se distribuyen los errores del modelo entre las distintas clases. Fashion-MNIST es un buen lugar para ver por qué esto importa: algunas clases (Pantalón, Sandalia, Bolso) se ven muy distintas de todo lo demás, mientras que otras (Camisa, Camiseta/Top, Suéter, Abrigo) se ven similares incluso para el ojo humano. Un modelo puede reportar un accuracy general aceptable mientras falla silenciosamente en un puñado de clases visualmente confundibles — el accuracy por sí solo no te lo dirá.

Los cuatro resultados posibles para una sola clase

Para cualquier clase (digamos, detectar "Camisa"), cada predicción cae en una de cuatro categorías:

Situación	Resultado
Realmente Camisa, Predicho Camisa	Verdadero Positivo (TP)
Realmente Camisa, Predicho NO Camisa	Falso Negativo (FN)
Realmente NO Camisa, Predicho Camisa	Falso Positivo (FP)
Realmente NO Camisa, Predicho NO Camisa	Verdadero Negativo (TN)

Recall — ¿capturamos los casos reales?

Recall pregunta: de todas las muestras que realmente pertenecen a esta clase, ¿cuántas identificó correctamente el modelo?

$$\text{Recall} = \text{TP} / (\text{TP} + \text{FN})$$

Solo observa las muestras cuya etiqueta verdadera es "Camisa" — de todo lo que realmente era una camisa, ¿qué fracción encontró el modelo? Un recall bajo en una clase significa que el modelo está perdiendo instancias reales de ella, etiquetándolas como otra cosa.

Precision — ¿nuestras predicciones positivas fueron realmente correctas?

Precision plantea una pregunta distinta: de todo lo que el modelo etiquetó como "Camisa", ¿cuánto realmente era camisa?

$$\text{Precision} = \text{TP} / (\text{TP} + \text{FP})$$

Esto observa las predicciones del modelo, no las etiquetas verdaderas. Una precision baja significa que el modelo está etiquetando otras cosas como "Camisa" con demasiada facilidad.

Por qué necesitas ambas: Un modelo podría lograr recall perfecto prediciendo "Camisa" para todo — nunca perdería una camisa real, pero la precision sería pésima. En cambio, un modelo podría lograr precision perfecta prediciendo "Camisa" solo cuando está extremadamente seguro en un caso obvio — pero perdería la mayoría de las camisas reales. Ninguno de los dos extremos es un modelo genuinamente bueno; necesitas ambas métricas juntas para saberlo.

F1-Score — equilibrando ambas en un solo número

F1 es la media armónica de precision y recall, combinadas en un solo puntaje:

$$\text{F1} = 2 * (\text{Precision} * \text{Recall}) / (\text{Precision} + \text{Recall})$$

Es específicamente la media armónica, no un promedio simple, porque eso castiga el desequilibrio con más severidad: si precision o recall es muy bajo, F1 se arrastra hacia ese valor bajo en lugar de ser elevado por el otro. Esto hace de F1 un resumen útil en un solo número cuando una clase necesita manejarse bien en ambos frentes — ser realmente capturada, y no sobre-predicha.

TAREA — 3.7 — Calcula un Reporte por Clase para tu CNN

scikit-learn ofrece un reporte de una sola línea que cubre precision, recall, y F1-score para cada clase a la vez.

```
from sklearn.metrics import classification_report

# y_true: etiquetas verdaderas como arreglo de NumPy o lista de enteros
# y_pred: índices de clase predichos (por ejemplo, de torch.argmax(preds, dim=1))
print(classification_report(y_true, y_pred))
```

- Ejecuta esto sobre las predicciones de prueba de tu CNN en Fashion-MNIST.
- Identifica la clase (o clases) con el F1-score más bajo, y la(s) clase(s) con el más alto.

Pregunta de discusión: ¿Qué clases confunde más tu modelo? Observa específicamente Camisa, Camiseta/Top, Suéter, y Abrigo — ¿puntúan notablemente más bajo que clases como Pantalón, Sandalia, o Bolso? ¿Esto coincide con lo que esperarías simplemente al ver algunas imágenes de ejemplo de cada clase?

Pregunta de discusión: Si tu accuracy general es, digamos, 90%, pero una clase tiene un F1-score bastante por debajo de eso, ¿qué oculta el número de accuracy general? ¿Por qué el accuracy general

por sí solo podría ser una forma engañosa de decidir si un modelo es "suficientemente bueno" para desplegar?

Pregunta de discusión: ¿Esperarías que el data augmentation (rotación, traslación) ayude a distinguir Camisa de Camiseta/Top? ¿Qué tipo de confusión es esa — un problema de traslación/orientación, o algo distinto? ¿Qué implica tu respuesta sobre qué técnicas probablemente ayuden con qué tipos de errores?

Parte 4 — Experimentando con Dropout

Agrega una capa Dropout a tu CNN de Fashion-MNIST de la Parte 3, aplicada después de aplanar y antes de la capa densa final.

```
self.dropout = nn.Dropout(p=0.____)

# en forward(), después de aplanar:
output = output.view(output.size(0), -1)
output = self.dropout(output)
output = self.fc_out(output)
```

TAREA — 4.1 — Ejecuta un Pequeño Barrido de Dropout

Usando la misma semilla fija y división de datos del resto de este laboratorio, entrena tu CNN de Fashion-MNIST con al menos 4 tasas de dropout distintas (por ejemplo, 0.0, 0.2, 0.4, 0.6) y registra la precisión de prueba para cada una.

dropout_p	Precisión de Prueba
0.0	
0.2	
0.4	
0.6	

TAREA — 4.2 — Analiza Tus Resultados

- ¿El dropout cambia de forma significativa la precisión de prueba en este caso, o la diferencia está dentro del margen de ruido que esperarías de una sola ejecución?
- ¿Se reduce la brecha entre la pérdida de entrenamiento y la de validación conforme aumenta el dropout, aunque la precisión de prueba no cambie mucho?
- Fashion-MNIST tiene 60,000 imágenes de entrenamiento, frente a ~455 en el dataset de cáncer de mama de tu laboratorio anterior. ¿Cambia esto cuánto esperarías que el dropout importe aquí? ¿Por qué sí o por qué no?
- Dada la brecha que mediste entre la Parte 2 y la Parte 3, ¿el dropout cierra más de esa brecha restante, o el cambio de arquitectura (densa -> CNN) sigue explicando la mayor parte de la mejora?

Entregables

- Un único notebook que contenga: la carga de datos para ambos datasets, el modelo denso (entrenado tanto en MNIST como en Fashion-MNIST), el modelo CNN, tu barrido de dropout, el reporte de clasificación por clase, y todas las gráficas solicitadas.
- Un breve reporte escrito (una página es suficiente) respondiendo, con tus propias palabras, cada pregunta de discusión planteada a lo largo de este laboratorio.
- Las tablas de comparación completadas de las Partes 2, 3 y 4, más el reporte de precision/recall/F1 por clase de tu CNN de la Parte 3.

Metas Adicionales (Opcional)

- Agrega una tercera capa convolucional a tu CNN de Fashion-MNIST y observa si la precisión mejora aún más, se estanca, o empeora.
- Visualiza algunas imágenes de prueba de Fashion-MNIST mal clasificadas junto con la predicción del modelo frente a la etiqueta verdadera — ¿los errores parecen razonables para un humano (por ejemplo, camisa vs. abrigo), o son extraños?
- Ahora ejecuta tu CNN también en MNIST, y compárala con tu resultado denso de la Parte 1 — ¿la ventaja de la CNN sobre la red densa se reduce en el dataset más fácil? ¿Qué te dice eso sobre cuándo importan más las mejoras arquitectónicas?